

# Lecture 3: Introduction to the Tidyverse

ØKA2015 Applied Data Science, University of Inland Norway

Touseef Hameed

16 September 2026

## Contents

|    |                                                     |   |
|----|-----------------------------------------------------|---|
| 1  | What this document is                               | 2 |
| 2  | R as a calculator                                   | 2 |
| 3  | Objects: the assignment operator <code>&lt;-</code> | 2 |
| 4  | Vectors: <code>c()</code>                           | 3 |
| 5  | Functions and arguments                             | 3 |
| 6  | Comments and object names                           | 4 |
| 7  | Precision: R does exactly what you type             | 4 |
| 8  | Packages: install once, load every session          | 4 |
| 9  | Reading a CSV file with <code>readr</code>          | 5 |
| 10 | A first look at the dataset                         | 5 |
| 11 | Real data is rarely clean                           | 7 |
| 12 | Saving a copy, and other file types                 | 7 |
| 13 | What you can do now                                 | 8 |
| 14 | Sources                                             | 8 |

# 1 What this document is

This is an **R Markdown** file. It combines ordinary text with chunks of R code. When you *knit* it, R runs every chunk and places the results into a PDF or a web page. It is the form your assignments take: the reader sees your code, its output and your explanation together.

The same code, without the surrounding text, is in `lecture03_tidyverse.R`. Open that file in RStudio and run it line by line with **Ctrl + Enter**. Every line there carries a comment explaining what it does.

## 2 R as a calculator

R evaluates whatever you type at the console.

```
1 + 1          # add
```

```
#> [1] 2
```

```
1 / 200 * 30   # divide, then multiply
```

```
#> [1] 0.15
```

```
(59 + 73 + 2) / 3 # parentheses first: an average
```

```
#> [1] 44.66667
```

The `[1]` at the start of each answer indicates that the line of output begins at element 1 of the result. It becomes meaningful once results contain several elements.

## 3 Objects: the assignment operator `<-`

An **object** is a named location in R's memory that holds a value. The operator `<-` places a value into it. Every assignment has the form `object_name <- value`; read it as “*object name gets value*”. In RStudio the shortcut for `<-` is **Alt + minus**.

The running example is the coffee bar from Lecture 2, where a latte costs 52 NOK.

```
cups <- 18      # store 18 under the name cups; nothing is printed  
cups           # the name alone displays the value
```

```
#> [1] 18
```

```
price <- 52     # the price of one latte, in NOK  
cups * price    # revenue for the day
```

```
#> [1] 936
```

## 4 Vectors: `c()`

A **vector** holds several values of the same kind in one object. `c()` means *combine*. R applies an operation to every element of a vector at once, which is called vectorised computation; it is why R code rarely needs a loop.

```
cups <- c(18, 22, 27)      # Monday, Tuesday, Wednesday
cups * price               # the price applied to every element
```

```
#> [1]  936 1144 1404
```

```
sum(cups)                  # total cups over three days
```

```
#> [1] 67
```

```
mean(cups)                 # average per day
```

```
#> [1] 22.33333
```

Text is written in quotation marks; numbers are not.

```
days <- c("Mon", "Tue", "Wed") # a vector of text
days
```

```
#> [1] "Mon" "Tue" "Wed"
```

## 5 Functions and arguments

A **function** is a named command: `function_name(argument1 = value1, argument2 = value2)`. Arguments sit inside the parentheses, separated by commas. The first few may be given by position; later ones are clearer by name.

```
seq(from = 1, to = 10)     # arguments by name
```

```
#> [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(1, 10)                 # the same, by position
```

```
#> [1]  1  2  3  4  5  6  7  8  9 10
```

```
round(mean(cups), 1)       # a function inside a function
```

```
#> [1] 22.3
```

```
mean(c(18, NA, 27), na.rm = TRUE)    # NA is a missing value; na.rm = TRUE ignores it
```

```
#> [1] 22.5
```

## 6 Comments and object names

Everything after `#` on a line is a **comment**: R ignores it, and it explains the code to a human reader. In your assignments, a comment above every command is required.

A name must start with a letter and may contain letters, numbers, `_` and `.` only. Use descriptive names in one consistent style; this course uses **snake\_case**.

```
cups_monday <- 18      # snake_case, recommended  
cupsMonday  <- 18      # camelCase, also valid
```

## 7 Precision: R does exactly what you type

1. **R is case-sensitive.** `cups`, `Cups` and `CUPS` are three different names. The error message always names the object it could not find.
2. **Text requires quotation marks; numbers do not.** Writing `town <- Lillehammer` makes R search for an *object* called `Lillehammer`, and fail.
3. **Quotation marks and parentheses come in pairs.** If the console shows `+` and waits, a closing `"` or `)` is missing. Complete it, or press **Esc**.

## 8 Packages: install once, load every session

A **package** is a collection of functions, data and documentation written by others. The **tidyverse** is the collection of packages this course is built on. Two commands, and they are not interchangeable:

- `install.packages("tidyverse")` **downloads** it to your laptop. Once per computer. With quotation marks.
- `library(tidyverse)` **loads** it for the current session. Every time you open R. Without quotation marks.

```
install.packages("tidyverse")    # run once on each computer you use
```

```
library(tidyverse)              # load the nine core packages for this session
```

When it loads, R lists nine core packages. A message about *conflicts* is information, not an error: two packages define a function with the same name, and the tidyverse version takes precedence. `tidyverse_update()` reports whether newer versions exist.

If you ever see `there is no package called 'x'`, it means `install.packages("x")` has not been run on that computer.

## 9 Reading a CSV file with readr

`read_csv()` reads a comma-separated file and returns a **tibble**, the tidyverse's data table. A tibble prints its dimensions, the type of every column, and only the first ten rows, so a large dataset remains readable.

The example file has six students and five columns, from *R for Data Science*. There are three routes into R; the first argument, the path, is the most important one.

**Route 1, from the web.** Nothing to download first.

```
students <- read_csv("https://pos.it/r4ds-students-csv") # the path is a web address
```

**Route 2, from a file on your laptop.** The path is relative to your *working directory*, the folder R reads from and writes to. `getwd()` reports it. This document uses route 2 so that it knits without an internet connection.

```
students <- read_csv("data/students.csv") # relative to the working directory
```

**Route 3, the Import button.** Environment pane, *Import Dataset > From Text (readr)*, choose the file, and RStudio writes route 2 for you. Copy the command it shows into your script so that the import is reproducible.

## 10 A first look at the dataset

Always inspect before you analyse. These commands are Assignment 1, Problems 1 to 4.

```
students # dimensions, column names, column types
```

```
#> # A tibble: 6 x 5
#>   'Student ID' 'Full Name' favourite.food mealPlan AGE
#>   <dbl> <chr> <chr> <chr> <chr>
#> 1 1 Sunil Huffmann Strawberry yoghurt Lunch only 4
#> 2 2 Barclay Lynn French fries Lunch only 5
#> 3 3 Jayendra Lyne N/A Breakfast and lunch 7
#> 4 4 Leon Rossini Anchovies Lunch only <NA>
#> 5 5 Chidiegwu Dunkel Pizza Breakfast and lunch five
#> 6 6 Güvenç Attila Ice cream Lunch only 6
```

```
head(students) # first six rows
```

```
#> # A tibble: 6 x 5
#>   'Student ID' 'Full Name' favourite.food mealPlan AGE
#>   <dbl> <chr> <chr> <chr> <chr>
#> 1 1 Sunil Huffmann Strawberry yoghurt Lunch only 4
#> 2 2 Barclay Lynn French fries Lunch only 5
#> 3 3 Jayendra Lyne N/A Breakfast and lunch 7
```

```
#> 4          4 Leon Rossini    Anchovies    Lunch only    <NA>
#> 5          5 Chidiegwu Dunkel Pizza      Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream    Lunch only    6
```

```
nrow(students)    # observations
```

```
#> [1] 6
```

```
ncol(students)    # variables
```

```
#> [1] 5
```

```
names(students)    # column names
```

```
#> [1] "Student ID"      "Full Name"        "favourite.food" "mealPlan"
#> [5] "AGE"
```

```
glimpse(students)    # every column on its own line, with its type
```

```
#> Rows: 6
#> Columns: 5
#> $ 'Student ID'      <dbl> 1, 2, 3, 4, 5, 6
#> $ 'Full Name'       <chr> "Sunil Huffmann", "Barclay Lynn", "Jayendra Lyne", "Leo~
#> $ favourite.food    <chr> "Strawberry yoghurt", "French fries", "N/A", "Anchovies~
#> $ mealPlan          <chr> "Lunch only", "Lunch only", "Breakfast and lunch", "Lun~
#> $ AGE               <chr> "4", "5", "7", NA, "five", "6"
```

```
summary(students)    # basic statistics per column
```

```
#>   Student ID      Full Name      favourite.food      mealPlan
#> Min.   :1.00      Length:6      Length:6      Length:6
#> 1st Qu.:2.25      Class :character  Class :character  Class :character
#> Median :3.50      Mode  :character  Mode  :character  Mode  :character
#> Mean    :3.50
#> 3rd Qu.:4.75
#> Max.    :6.00
#>      AGE
#> Length:6
#> Class :character
#> Mode  :character
#>
#>
#>
```

Note the type under each column name: `<chr>` is text, `<dbl>` is a number. `AGE` was read as text. That is a clue.

## 11 Real data is rarely clean

Two features of this file are typical of real data.

“N/A” is text, not a missing value. R treats only a genuinely empty cell as missing (NA). The string "N/A" is three characters of text. State which strings mean missing:

```
students <- read_csv("data/students.csv", na = c("N/A", "")) # na = : a named argument
students$favourite.food # $ selects one column; row 3 is now NA
```

```
#> [1] "Strawberry yoghurt" "French fries"      NA
#> [4] "Anchovies"         "Pizza"             "Ice cream"
```

```
sum(is.na(students)) # is.na() is TRUE where missing; sum() counts them
```

```
#> [1] 2
```

```
colSums(is.na(students)) # per column
```

```
#>      Student ID      Full Name favourite.food      mealPlan      AGE
#>           0           0           1           0           1
```

“five” is a word, not a number. One age is written out, so the whole column became text. Replace the word and convert the column:

```
students$AGE <- gsub("five", "5", students$AGE) # global substitution
students$AGE <- as.numeric(students$AGE)       # text to numbers
students$AGE
```

```
#> [1] 4 5 7 NA 5 6
```

```
mean(students$AGE, na.rm = TRUE) # ignoring the missing value
```

```
#> [1] 5.4
```

Lecture 4 performs this kind of correction with the dplyr verbs. This is a preview of why data wrangling takes 50 to 80 per cent of a data scientist’s time.

## 12 Saving a copy, and other file types

write\_csv() is the reverse of read\_csv(): the object, then the path.

```
dir.create("data", showWarnings = FALSE)      # make the folder if it does not exist
write_csv(students, "data/students_clean.csv") # write the tibble to disk
file.exists("data/students_clean.csv")        # TRUE if the file is there
```

```
#> [1] TRUE
```

Several files can be read in one call, with `id = "file"` recording the source of each row:

```
sales_files <- c("https://pos.it/r4ds-01-sales",
                 "https://pos.it/r4ds-02-sales",
                 "https://pos.it/r4ds-03-sales")
sales <- read_csv(sales_files, id = "file")
```

Other formats use the same idea with a different function: `read_csv2()` for semicolon-separated files, which is common in Norway because the comma is the decimal marker; `read_tsv()` for tab-separated files; `read_delim()` for any delimiter; `readxl::read_excel()` for Excel workbooks.

## 13 What you can do now

- store values in objects and combine them with `c()`
- call a function with named or positional arguments
- install and load a package
- read a CSV from the web or from disk
- inspect its dimensions, names, types and summary
- recognise the two most common ways real data arrives incorrect

**Lecture 4, Friday 18 September:** the dplyr verbs `filter()`, `select()`, `mutate()`, `arrange()`, `group_by()` and `summarise()`. That is Assignment 1, Problems 5 and 6.

## 14 Sources

- Wickham, Çetinkaya-Rundel and Golemund (2023), *R for Data Science*, 2nd ed. Introduction (*Prerequisites*), Ch. 2 *Workflow: basics*, Ch. 7 *Data import*. Free at <https://r4ds.hadley.nz>.
- The tidyverse: <https://www.tidyverse.org>
- readr: <https://readr.tidyverse.org>